

EXP1

```
!pip install scikit-fuzzy
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

temperature = ctrl.Antecedent(np.arange(0, 101, 1), 'temperature')
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')

temperature['low'] = fuzz.trimf(temperature.universe, [0, 0, 50])
temperature['medium'] = fuzz.trimf(temperature.universe, [25, 50, 75])
temperature['high'] = fuzz.trimf(temperature.universe, [50, 100, 100])

fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0, 0, 50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [25, 50, 75])
fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50, 100, 100])

rule1 = ctrl.Rule(temperature['low'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['medium'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['high'], fan_speed['high'])

fan_control_system = ctrl.ControlSystem([rule1, rule2, rule3])
fan_speed_simulation = ctrl.ControlSystemSimulation(fan_control_system)

temperature_value = 55
fan_speed_simulation.input['temperature'] = temperature_value

fan_speed_simulation.compute()

print("Input Temperature:", temperature_value)
print("Fan Speed Output:", fan_speed_simulation.output['fan_speed'])

temperature.view()
fan_speed.view()

fan_speed.view(sim=fan_speed_simulation)
```

EXP2

```
import numpy as np
```

```
class DiscretePerceptron:
```

```
def __init__(self, input_size):
```

```
    self.weights = np.zeros(input_size)
```

```
    self.bias = 0
```

```
def predict(self, inputs):
```

```
    activation = np.dot(self.weights, inputs) + self.bias
```

```
    return 1 if activation > 0 else 0
```

```
def train(self, inputs, target, learning_rate=0.1, epochs=100):
```

```
    for _ in range(epochs):
```

```
        for x, y in zip(inputs, target):
```

```
            prediction = self.predict(x)
```

```
            error = y - prediction
```

```
            self.weights += learning_rate * error * x
```

```
            self.bias += learning_rate * error
```

```
def main():
```

```
    class_0 = np.array([[2, 3], [3, 2], [1, 1]])
```

```
    class_1 = np.array([[5, 7], [6, 8], [7, 6]])
```

```
    inputs = np.vstack((class_0, class_1))
```

```
    targets = np.array([0, 0, 0, 1, 1, 1])
```

```
    perceptron = DiscretePerceptron(input_size=2)
```

```
    perceptron.train(inputs, targets)
```

```
    test_data = np.array([[4, 5], [2, 2]])
```

```
    for data in test_data:
```

```
        prediction = perceptron.predict(data)
```

```
        if prediction == 0:
```

```
            print(f"Data {data} belongs to class 0")
```

```
        else:
```

```
            print(f"Data {data} belongs to class 1")
```

```
if __name__ == "__main__":
```

```
    main()
```

EXP3

```
import numpy as np
```

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):  
    return x * (1 - x)  
input_data = np.array([  
    [0, 0],  
    [0, 1],  
    [1, 0],  
    [1, 1]  
])
```

```
target_data = np.array([  
    [0],  
    [1],  
    [1],  
    [0]  
])  
input_size = 2  
hidden_size = 2  
output_size = 1
```

```
learning_rate = 0.1  
epochs = 10000
```

```
hidden_weights = np.random.uniform(size=(input_size, hidden_size))  
output_weights = np.random.uniform(size=(hidden_size, output_size))
```

```
for _ in range(epochs):  
    hidden_layer_activation = np.dot(input_data, hidden_weights)  
    hidden_layer_output = sigmoid(hidden_layer_activation)
```

```
    output_layer_activation = np.dot(hidden_layer_output, output_weights)  
    predicted_output = sigmoid(output_layer_activation)
```

```
    error = target_data - predicted_output
```

```
    output_delta = error * sigmoid_derivative(predicted_output)
```

```
    hidden_layer_error = output_delta.dot(output_weights.T)  
    hidden_layer_delta = hidden_layer_error * sigmoid_derivative(hidden_layer_output)
```

```
    output_weights += hidden_layer_output.T.dot(output_delta) * learning_rate  
    hidden_weights += input_data.T.dot(hidden_layer_delta) * learning_rate
```

```
test_data = np.array([
[0, 0],
[0, 1],
[1, 0],
[1, 1]
])

print("Final Outputs:")
for data in test_data:
    hidden_layer_activation = np.dot(data, hidden_weights)
    hidden_layer_output = sigmoid(hidden_layer_activation)

    output_layer_activation = np.dot(hidden_layer_output, output_weights)
    predicted_output = sigmoid(output_layer_activation)

    print(f"Input: {data} Predicted Output: {predicted_output[0]}")
```

EXP4

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
data = np.random.rand(100, 2)

grid_size = (10, 10)
input_dim = 2
learning_rate = 0.2
num_epochs = 1000

# Initialize SOM weight matrix
weight_matrix = np.random.rand(grid_size[0], grid_size[1], input_dim)

# Training SOM
for epoch in range(num_epochs):

    # Decay learning rate
    lr = learning_rate * (1 - epoch / num_epochs)

    # Neighborhood radius decay
    radius = max(grid_size) / 2 * (1 - epoch / num_epochs)

    for input_vector in data:

        # Find Best Matching Unit (BMU)
        distances = np.linalg.norm(weight_matrix - input_vector, axis=2)
        bmu_coords = np.unravel_index(np.argmin(distances), distances.shape)

        # Update weights
        for i in range(grid_size[0]):
            for j in range(grid_size[1]):

                # Distance from neuron to BMU
                distance_to_bmu = np.linalg.norm(
                    np.array([i, j]) - np.array(bmu_coords)
                )

                # Check neighborhood
                if distance_to_bmu <= radius:

                    # Influence function
                    influence = np.exp(
                        -(distance_to_bmu ** 2) / (2 * (radius ** 2))
                    )
```

```

# Weight update
weight_matrix[i, j] += (
influence * lr * (input_vector - weight_matrix[i, j])
)

# Create cluster map
cluster_map = np.zeros((grid_size[0], grid_size[1]))

for i in range(grid_size[0]):
for j in range(grid_size[1]):

distances = np.linalg.norm(data - weight_matrix[i, j], axis=1)
cluster_map[i, j] = np.argmin(distances)

# Visualization
plt.figure(figsize=(8, 8))

plt.pcolormesh(cluster_map, cmap='viridis', shading='auto')
plt.colorbar(label='Cluster Index')
# Plot data points
plt.scatter(
data[:, 0] * grid_size[0],
data[:, 1] * grid_size[1],
color='red',
label='Data Points'
)
plt.title("Self-Organizing Map (SOM) Clustering")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()

plt.show()

```

EXP5

```
import random
def fitness_function(x):
    return -x**2 + 6*x + 9
def initialize_population(pop_size, lower_bound, upper_bound):
    return [
        random.uniform(lower_bound, upper_bound)
        for _ in range(pop_size)
    ]
def select_parents(population):
    fitness_values = [fitness_function(ind) for ind in population]
    min_fitness = min(fitness_values)
    if min_fitness < 0:
        fitness_values = [f - min_fitness + 1 for f in fitness_values]
    parent1 = random.choices(population, weights=fitness_values)[0]
    parent2 = random.choices(population, weights=fitness_values)[0]
    return parent1, parent2

def crossover(parent1, parent2, crossover_prob=0.7):

    if random.random() < crossover_prob:

        # Arithmetic crossover
        alpha = random.random()

        child1 = alpha * parent1 + (1 - alpha) * parent2
        child2 = alpha * parent2 + (1 - alpha) * parent1

        return child1, child2

    else:
        return parent1, parent2

def mutate(individual, mutation_prob=0.01):

    if random.random() < mutation_prob:
        individual += random.uniform(-1, 1)

    return individual

def genetic_algorithm(generations, pop_size, lower_bound, upper_bound):

    population = initialize_population(
        pop_size,
        lower_bound,
        upper_bound
    )
```

```

for gen in range(generations):

    new_population = []

    while len(new_population) < pop_size:

        parent1, parent2 = select_parents(population)

        child1, child2 = crossover(parent1, parent2)

        child1 = mutate(child1)
        child2 = mutate(child2)

        new_population.extend([child1, child2])

    population = new_population[:pop_size]

    best_individual = max(population, key=fitness_function)

    print(
        f"Generation {gen+1}: "
        f"Best Individual = {best_individual:.4f}, "
        f"Fitness = {fitness_function(best_individual):.4f}"
    )

    return max(population, key=fitness_function)

if __name__ == "__main__":

    generations = 20
    pop_size = 100
    lower_bound = -10
    upper_bound = 10

    best_solution = genetic_algorithm(
        generations,
        pop_size,
        lower_bound,
        upper_bound
    )

    print("\nBest Solution Found:")
    print(f"x = {best_solution:.4f}")
    print(f"Fitness = {fitness_function(best_solution):.4f}")

```


EXP6

```
import random
import math

def fitness_function(x, y):
    return math.sin(x) + math.sin(y)

def initialize_population(pop_size, lower_bound, upper_bound):

    return [
        (
            random.uniform(lower_bound, upper_bound),
            random.uniform(lower_bound, upper_bound)
        )
        for _ in range(pop_size)
    ]

def select_parents(population):

    fitness_values = [
        fitness_function(x, y)
        for x, y in population
    ]

    min_fitness = min(fitness_values)

    if min_fitness < 0:
        fitness_values = [f - min_fitness + 1 for f in fitness_values]

    parent1 = random.choices(
        population,
        weights=fitness_values
    )[0]

    parent2 = random.choices(
        population,
        weights=fitness_values
    )[0]

    return parent1, parent2

def crossover(parent1, parent2, crossover_prob=0.7):

    if random.random() < crossover_prob:

        child1 = (parent1[0], parent2[1])
        child2 = (parent2[0], parent1[1])
```

```

return child1, child2

else:
return parent1, parent2

def mutate(individual, mutation_prob=0.01):

x, y = individual

if random.random() < mutation_prob:
x += random.uniform(-0.1, 0.1)

if random.random() < mutation_prob:
y += random.uniform(-0.1, 0.1)

return (x, y)

def genetic_algorithm(generations, pop_size, lower_bound, upper_bound):

population = initialize_population(
pop_size,
lower_bound,
upper_bound
)

for gen in range(generations):

new_population = []

while len(new_population) < pop_size:

parent1, parent2 = select_parents(population)

child1, child2 = crossover(parent1, parent2)

child1 = mutate(child1)
child2 = mutate(child2)

new_population.extend([child1, child2])

# Maintain population size
population = new_population[:pop_size]

# Best individual in current generation
best_individual = max(
population,

```

```

key=lambda ind: fitness_function(*ind)
)

print(
f"Generation {gen+1}: "
f"Best Individual = {best_individual}, "
f"Fitness = {fitness_function(*best_individual):.4f}"
)

# Final best solution
return max(
population,
key=lambda ind: fitness_function(*ind)
)

# Main Program
if __name__ == "__main__":

generations = 20
pop_size = 100

lower_bound = -2 * math.pi
upper_bound = 2 * math.pi

best_solution = genetic_algorithm(
generations,
pop_size,
lower_bound,
upper_bound
)

print("\nBest Solution Found:")
print(f"x = {best_solution[0]:.4f}")
print(f"y = {best_solution[1]:.4f}")
print(f"Fitness = {fitness_function(*best_solution):.4f}")

```